IoT ESP32 Project



- User interface
- Project technical details
- Interaction between project modules
- End-device architecture
- Data distribution in Amazon Web Services

Oleksii Pokormiakho



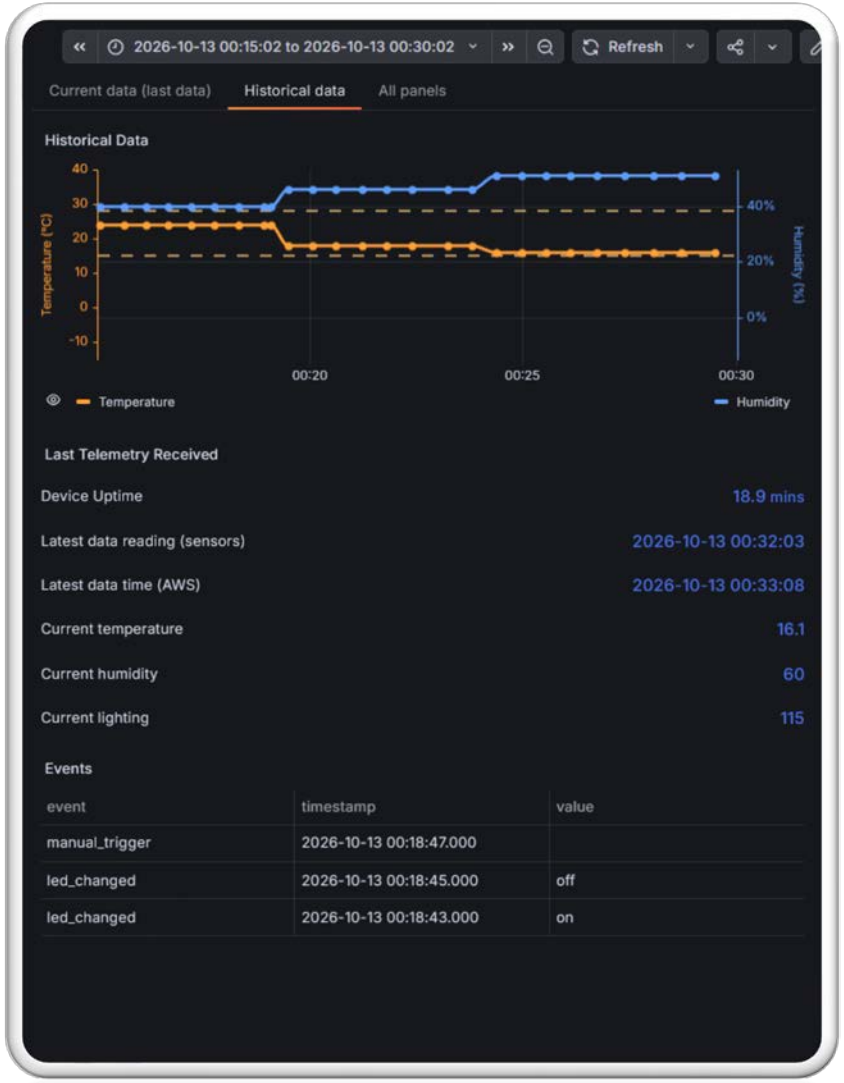
User interface

Grafana dashboard

Current data

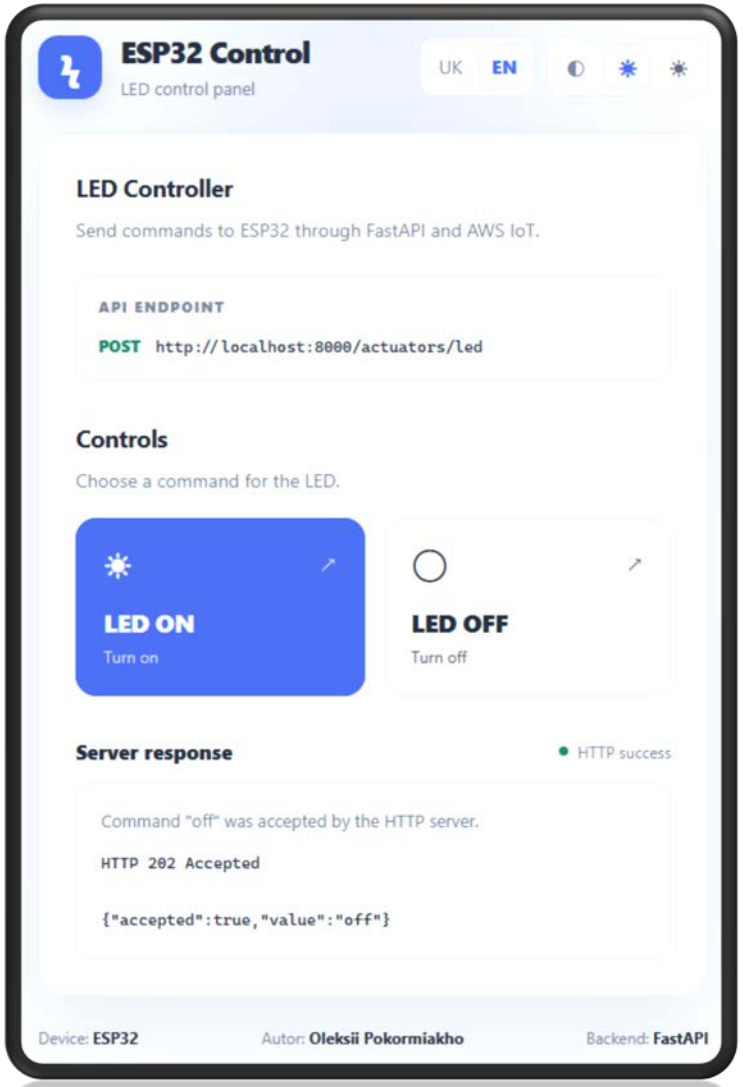


Historical data

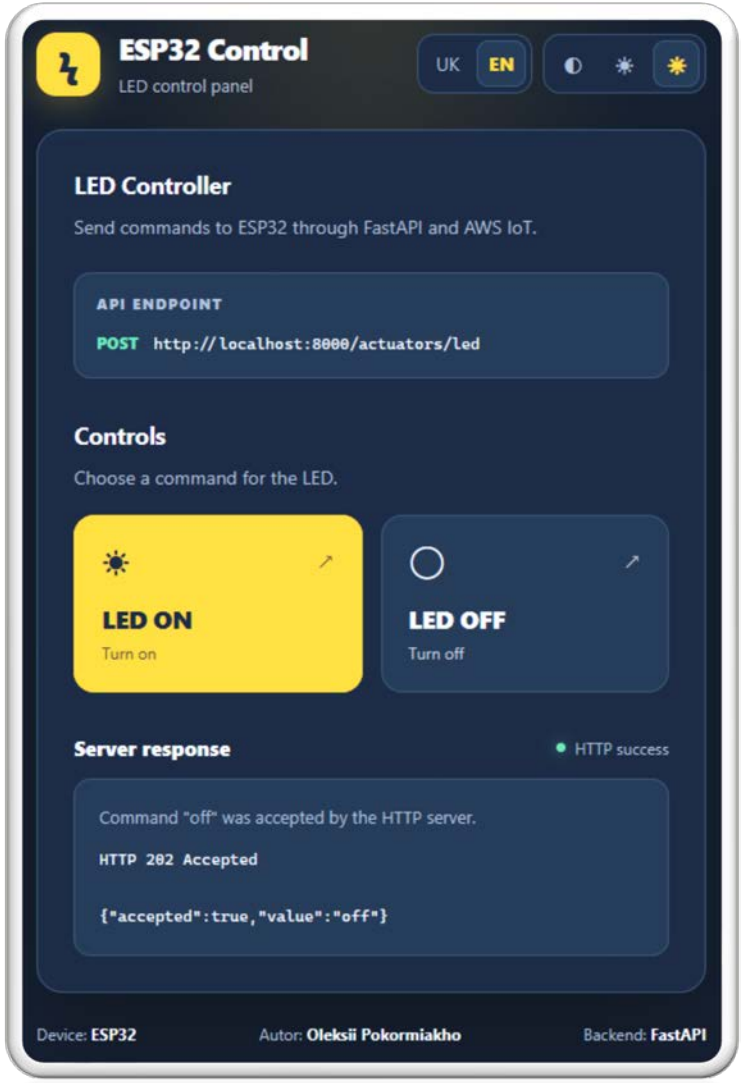


Web interface for device LED control

Light mode



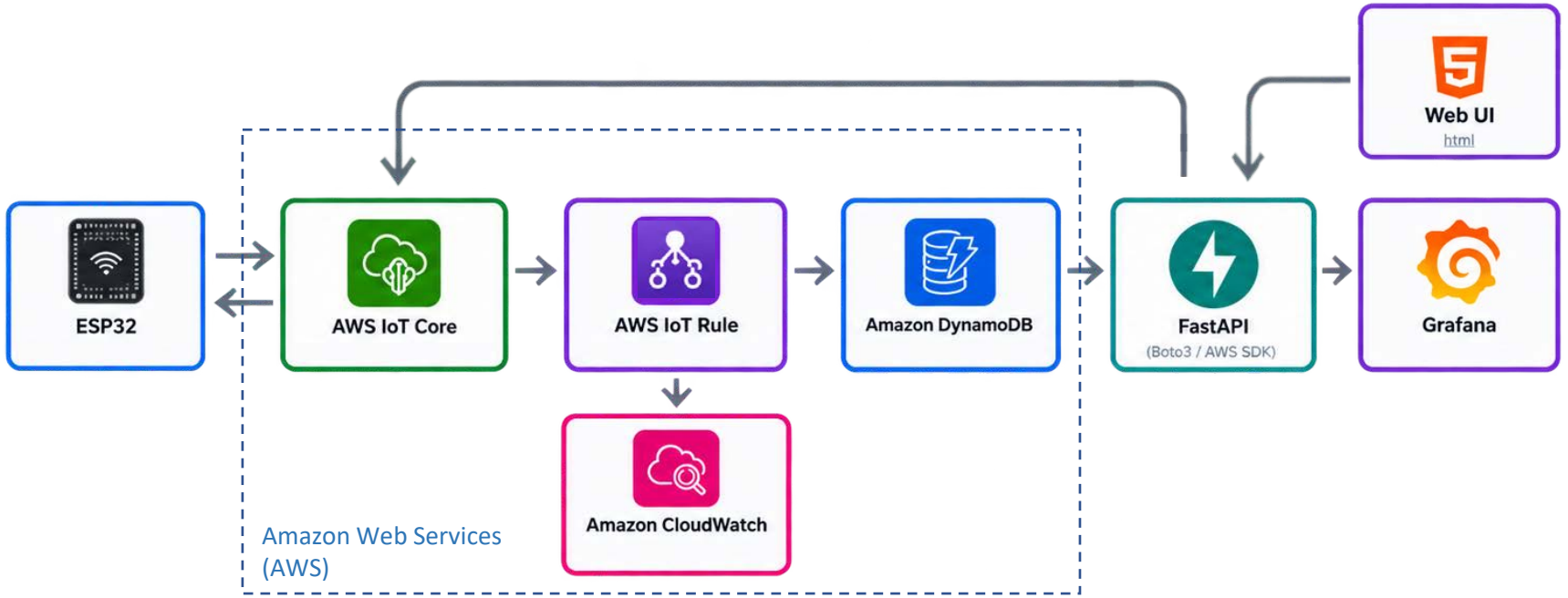
Dark mode





Project technical details

Simplified IoT data flow diagram



ESP32
Collects sensor data and sends it to AWS IoT Core using MQTT over TLS.
https://www.espressif.com

AWS IoT Core
Securely receives and manages device data from ESP32.
aws.amazon.com/iot-core

AWS IoT Rule
Filters and routes incoming IoT data to DynamoDB and CloudWatch.
aws.amazon.com/iot-rules

Amazon DynamoDB
Stores time-series data from IoT devices.
aws.amazon.com/dynamodb

FastAPI
Reads data from DynamoDB using Boto3 / AWS SDK and provides it to Grafana.
fastapi.tiangolo.com

Infinity Data Source Plugin
Connects Grafana to external data sources (e.g., FastAPI endpoint).
grafana.com/grafana/plugins

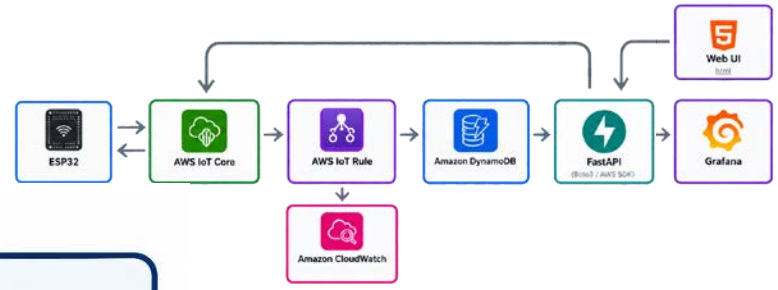
Grafana
Visualizes the data with dashboards and alerts.
grafana.com

Amazon CloudWatch
Collects and stores logs, metrics and events from AWS IoT Rule and other services.
aws.amazon.com/cloudwatch



Interaction between project modules

Interaction between ESP32 and Amazon Web Services (AWS)



ESP32



MQTT (publish)

MQTT (subscribe)

AWS

MQTT Topics (Subscribe)

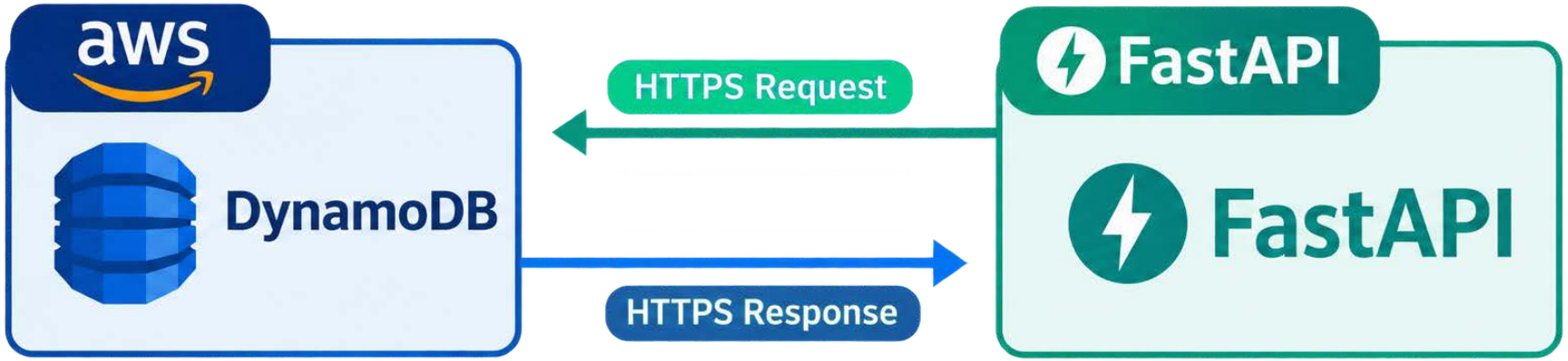
/ iot-course/OleksiiPok/commands/led

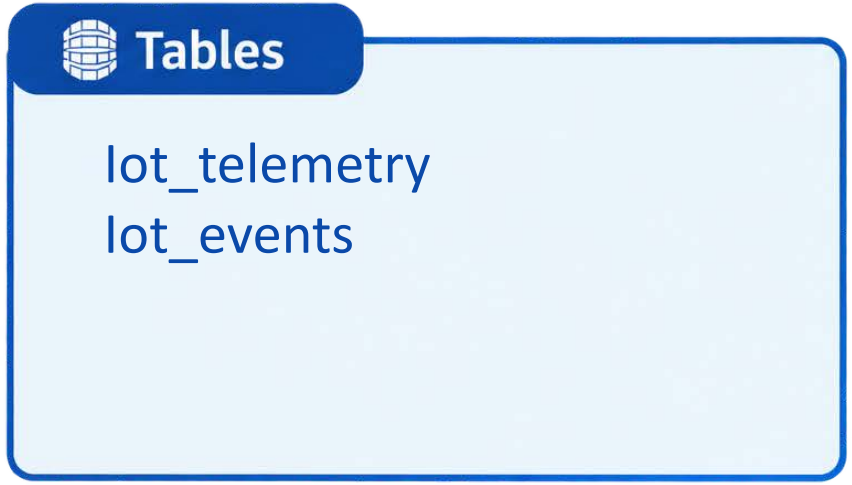
MQTT Topics (Publish)

/ iot-course/OleksiiPok/telemetry
/ iot-course/OleksiiPok/events
/ iot-course/OleksiiPok/commands

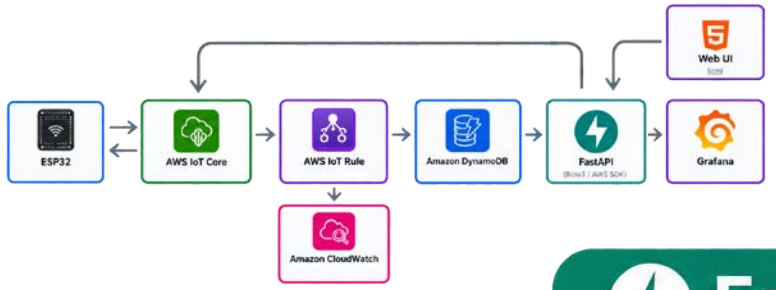
Interaction between Amazon DynamoDB and FastAPI







Interaction between FastAPI and Grafana



FastAPI

FastAPI

HTTP Request (REST API)

HTTP Response (JSON)

Grafana

Grafana

Endpoints

Resources:

Sensors/
Events/
Actuators/

Dashboard Panels

Tabs with panels:

Current Data
Historical Data

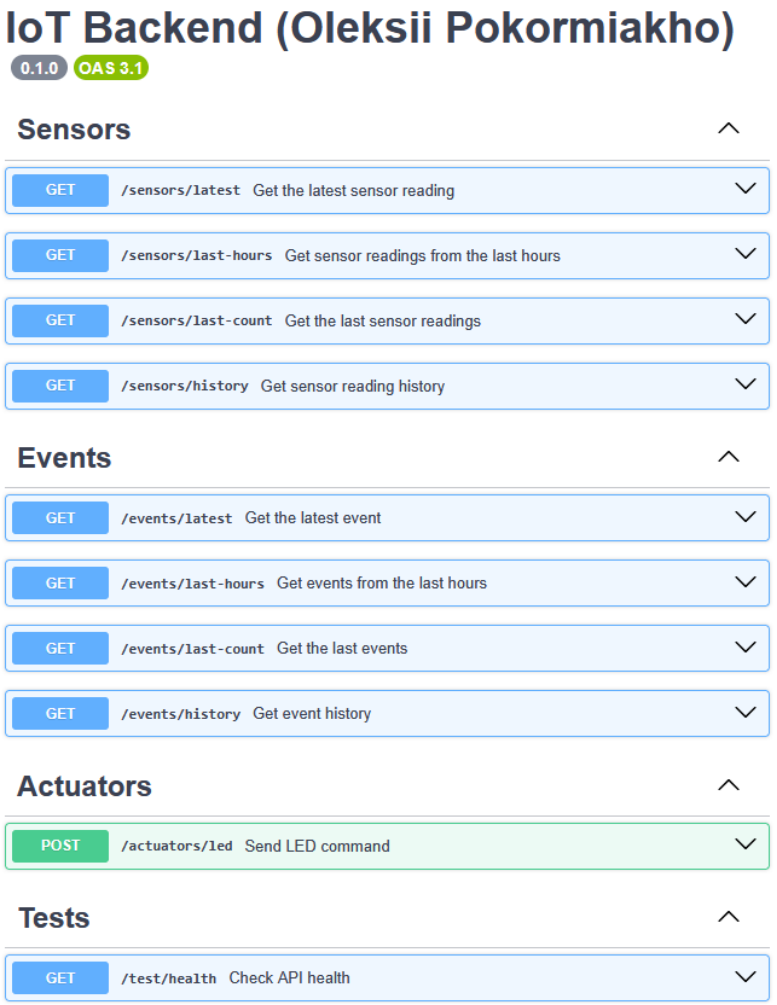
List of FastAPI endpoints

API Endpoints

GET /sensors/latest
GET /sensors/last-hours
GET /sensors/last-count
GET /sensors/history

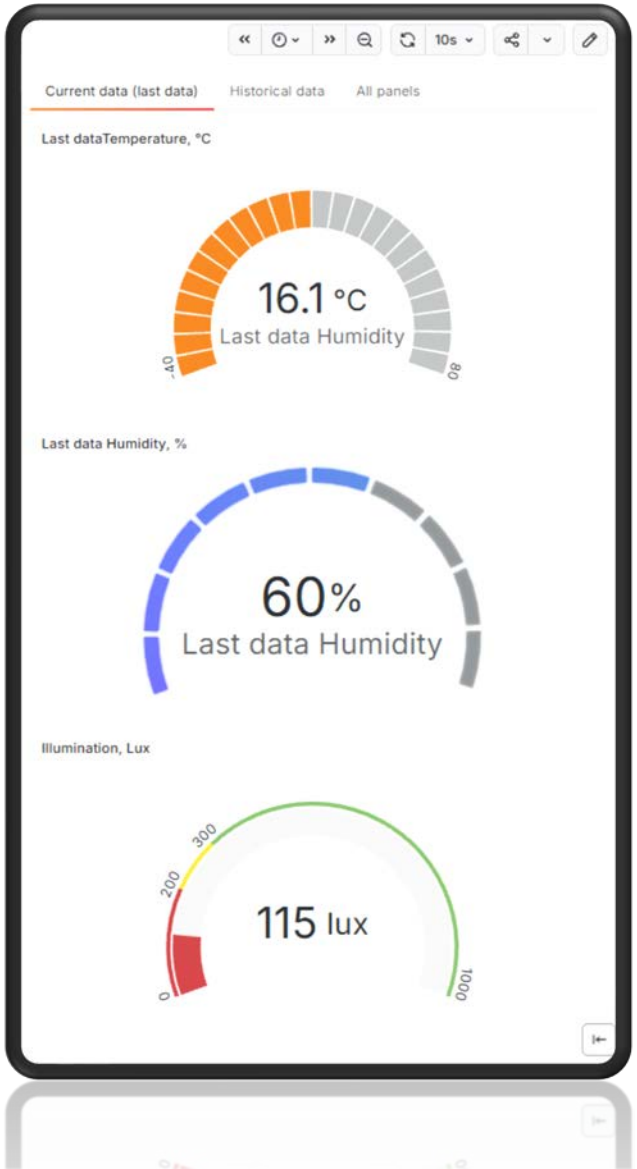
GET /events/latest
GET / events /last-hours
GET / events /last-count
GET / events /history

POST /actuators/led

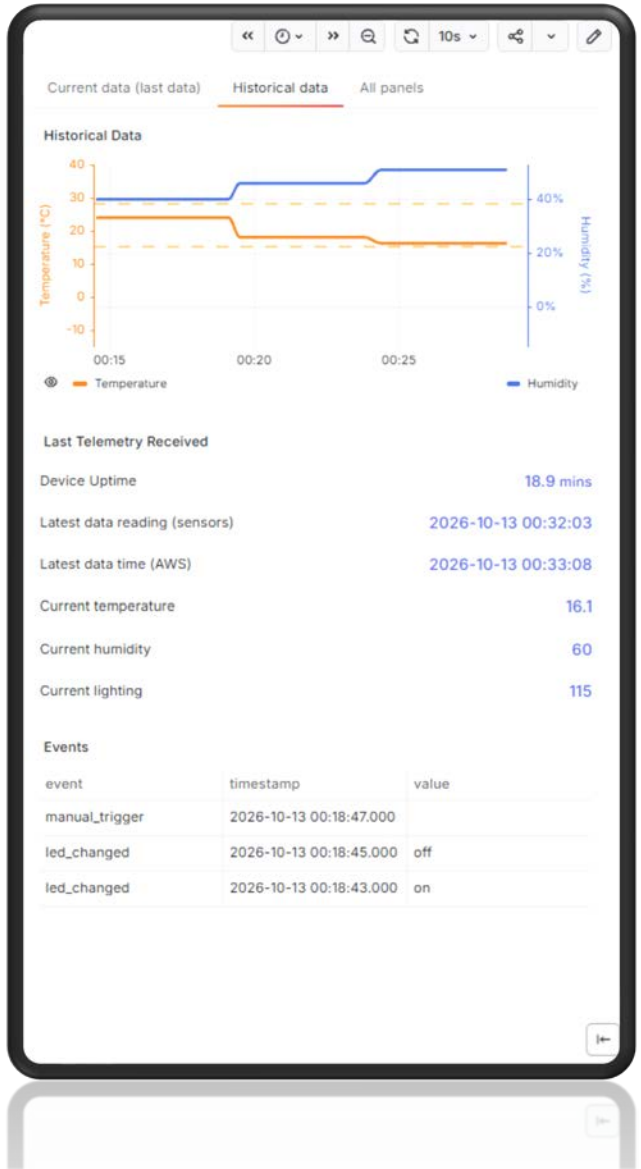


Grafana dashboard

Current data



Historical data





End-device architecture

End-device architecture

Software Architecture

ESP32 IoT Firmware

Sensors

DHT22
- Temperature & Humidity
- Read and validate data
- Alarm thresholds
- Status & STALE

LDR
- Light / Lux
- ADC reading & lux calculation
- Validate data
- Alarm thresholds
- Status & STALE

User Input

Buttons (4)
- Button 0 – manual_trigger
- Button 1 – silent mode
- Button 2 – light mode
- Button 3 – disconnect Wi-Fi

Core Modules

telemetry
- Collects device data
- Timestamps, uptime, sequence
- STALE watchdog

system_state
- System state bit-field
- Error aggregation
- State management

actions
- Handles commands
- LED state processing
- Device actions

indication
- LED control
- Patterns and states

Communication

wifi
- Wi-Fi connection management
- Reconnect logic
- Status monitoring

time
- NTP synchronization
- Timezone handling
- Timestamp provider

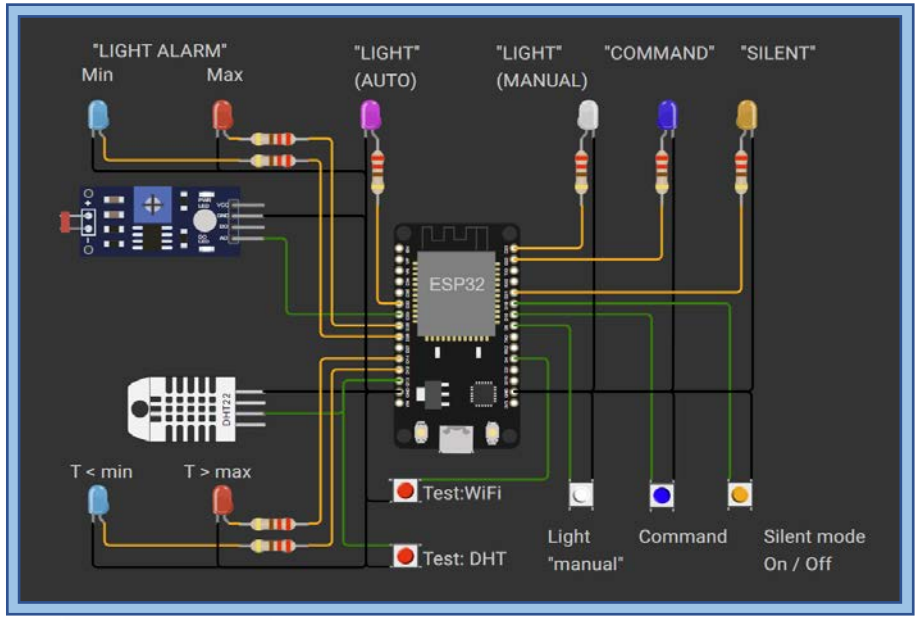
mqtt
- MQTT client
- Publish / Subscribe
- Message handling

serialization
- JSON serializers
- Telemetry / Status / Commands
- Events

Serial Monitor (Debug)
- Logs and diagnostics
- Sensor data
- System state
- MQTT payloads

Main Loop

Read sensors → Update state → Process actions → Communicate (MQTT) → Indicate (LED) → Monitor



Configuration Files

File	Purpose
app_config.h	Intervals and main settings
hardware_config.h	GPIO pins and hardware
dht_config.h	DHT22 limits and thresholds
ldr_config.h	LDR limits and thresholds
mqtt_config.h	MQTT service and topics
time_config.h	Timezone and NTP settings

Telemetry Data structure

Telemetry data

```
[MQTT] Published:
       "iot-course/OleksiiPok/telemetry"
[MQTT] JSON:
{
  "version": 4,
  "device_id": "39628656607504",
  "timestamp": 1791881464,
  "uptime": 59,
  "sequence": 1,
  "type": "telemetry",
  "dht": {
    "temperature": 17.1,
    "humidity": 60,
    "updated": 1791881464,
    "dht_status": 0
  },
  "ldr": {
    "raw": 1946,
    "lux": 114.9,
    "updated": 1791881464,
    "ldr_status": 64
  },
  "sys_state": 0
}
```

Status data

```
[DHT STATUS]
  STATUS BYTE: 0b00000000
               0b00000000
 ( 0) DEVICE_ERR              OFF
 ( 1) DATA_STALE              OFF
 ( 2) DATA_VALID_ERR          OFF
 ( 4) TEMPERATURE_ALARM_MIN   OFF
 ( 5) TEMPERATURE_ALARM_MAX   OFF
 ( 6) HUMIDITY_ALARM_MIN      OFF
 ( 7) HUMIDITY_ALARM_MAX      OFF

[LDR STATUS]
  STATUS BYTE: 0b00000000
               0b01000000
 ( 0) DEVICE_ERR              OFF
 ( 1) DATA_STALE              OFF
 ( 2) DATA_VALID_ERR          OFF
 ( 4) LUX_ALARM_MIN           OFF
 ( 5) LUX_ALARM_MAX           OFF
 ( 6) LIGHT_LOW               ON

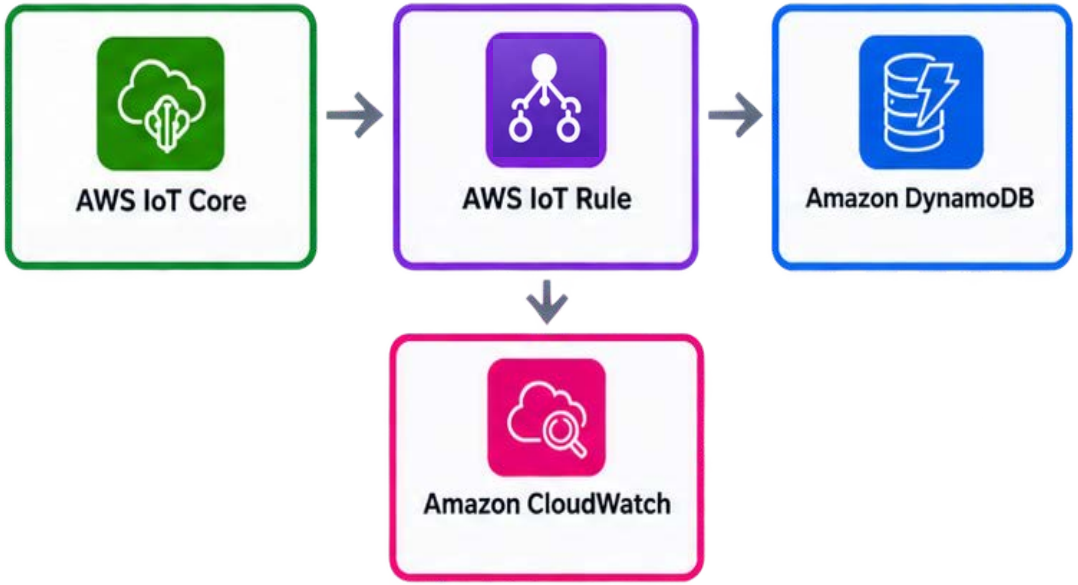
[SYSTEM STATE REGISTER]
  STATUS BYTE: 0b00000000
               0b00000000
 ( 0) SILENT_MODE             OFF
 ( 1) LDR_ERR                 OFF
 ( 2) DHT_ERR                 OFF
 ( 5) TIME_ERR                OFF
 ( 6) MQTT_ERR                OFF
 ( 7) WIFI_ERR                OFF
 ( 8) LIGHT                   OFF
```



Data distribution in Amazon Web Services

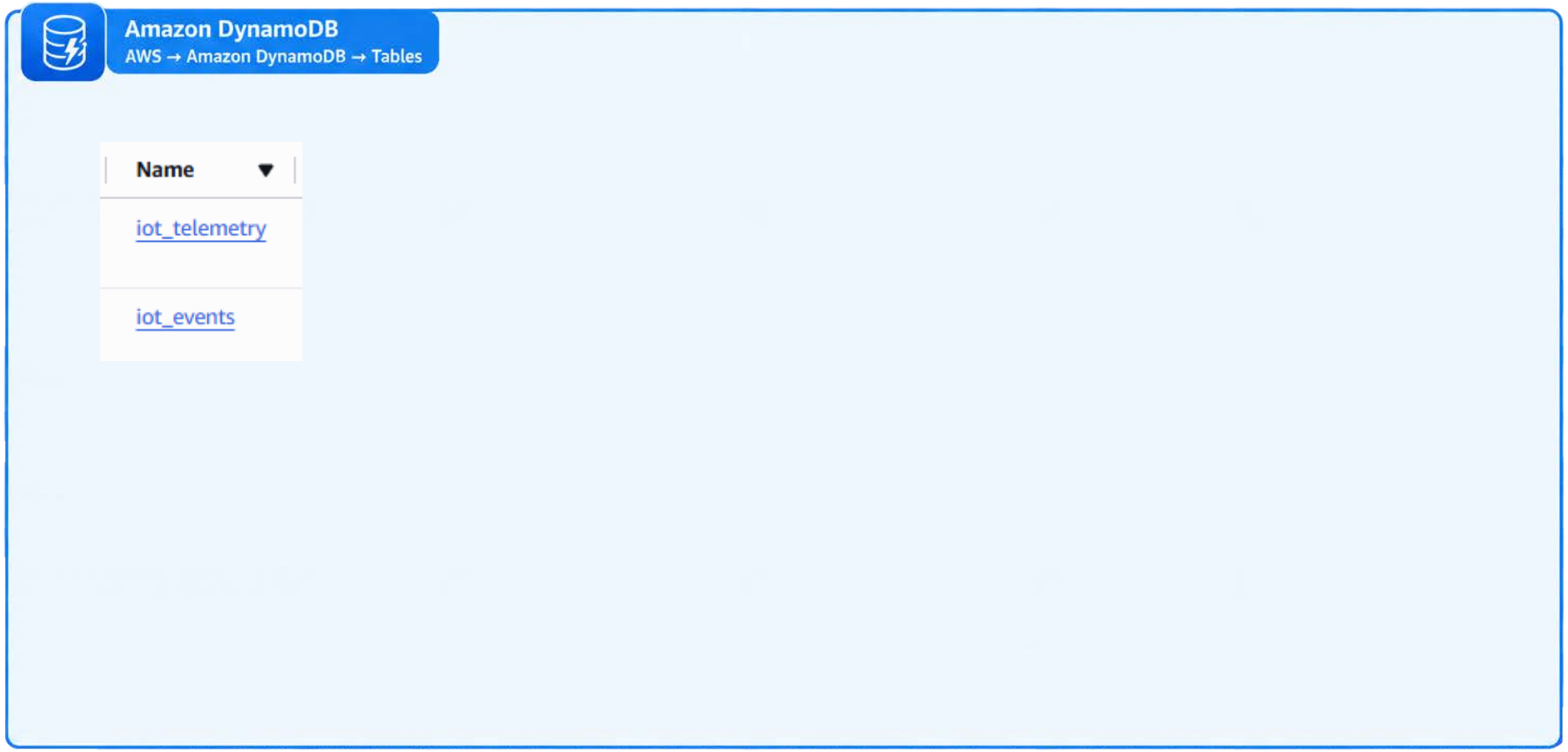
Amazon Web Services processing





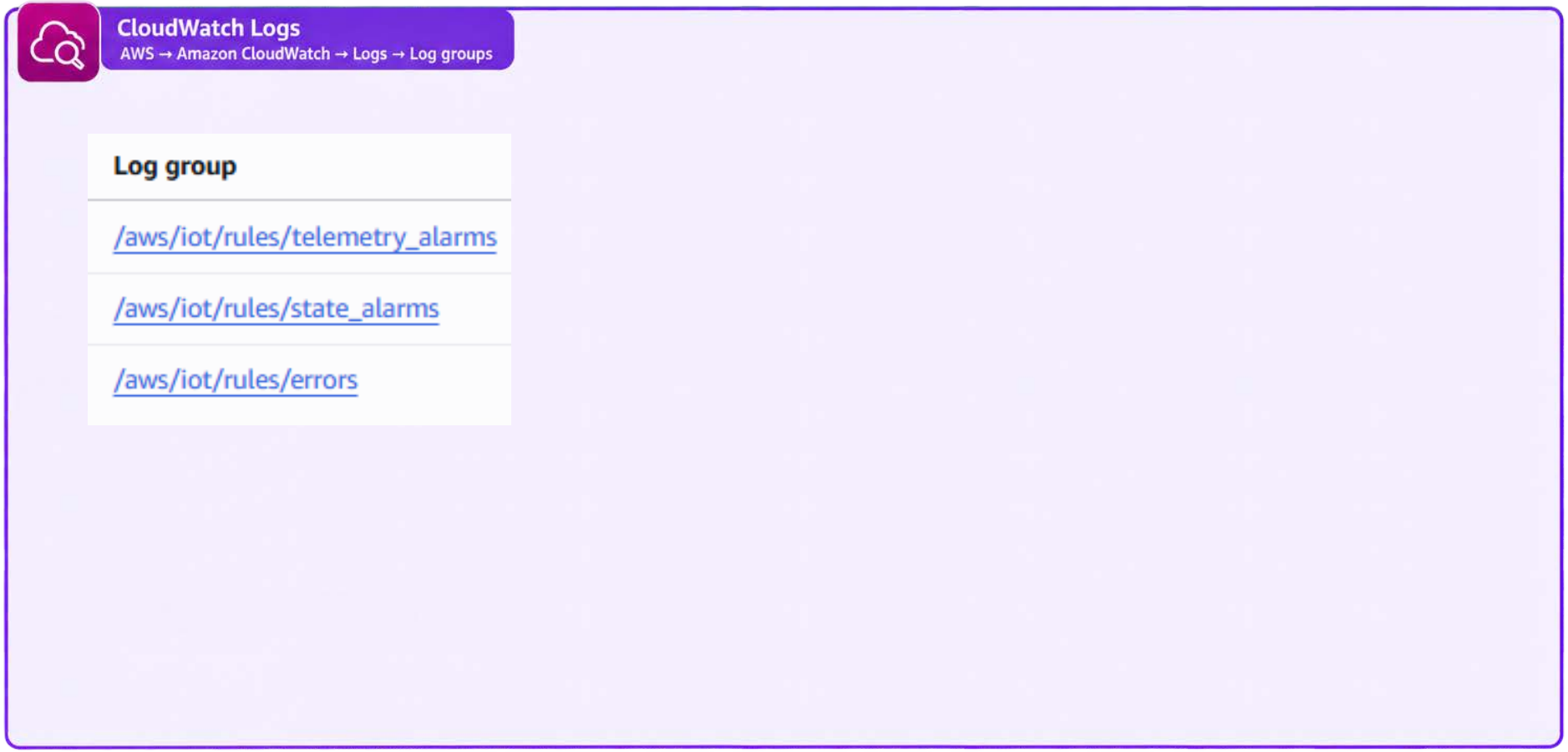
Amazon Web Services (AWS)





Amazon Web Services (AWS)





Amazon Web Services (AWS)



IoT Core Rules

AWS → AWS IoT Core → Message routing → Rules

Name	Status	Rule topic
temperature_alarm	Active	iot-course/OleksiiPok/telemetry
StoreTelemetry	Active	iot-course/OleksiiPok/telemetry
state_alarms	Active	iot-course/OleksiiPok/telemetry
events	Active	iot-course/OleksiiPok/events

```
SELECT *,
       timestamp() AS received_at,
       clientid()  AS client_id,
       topic(2)    AS student
FROM 'iot-course/OleksiiPok/telemetry'
WHERE bitand(sys_state, 2) = 0
  AND bitand(sys_state, 4) = 0
```

```
SELECT *,
       timestamp() AS received_at,
       CAST(clientid() AS String) AS device_id,
       topic(2)    AS student
FROM 'iot-course/OleksiiPok/events'
```

```
SELECT dht
FROM 'iot-course/OleksiiPok/telemetry'
WHERE bitand(sys_state, 2) = 0
AND bitand(sys_state, 4) = 0
AND (dht.temperature > 28 OR dht.temperature < 15)
```

```
SELECT *
FROM 'iot-course/OleksiiPok/telemetry'
WHERE sys_state <> 0
```

Amazon Web Services (AWS)

IoT Core Rules

AWS → AWS IoT Core → Message routing → Rules

Topic: .../telemetry

```
SELECT *,
       timestamp() AS received_at,
       clientid()  AS client_id,
       topic(2)    AS student
FROM 'iot-course/OleksiiPok/telemetry'
WHERE bitand(sys_state, 2) = 0
  AND bitand(sys_state, 4) = 0
```

→ DynamoDB /iot_telemetry

```
SELECT dht
FROM 'iot-course/OleksiiPok/telemetry'
WHERE bitand(sys_state, 2) = 0
AND bitand(sys_state, 4) = 0
AND (dht.temperature > 28 OR dht.temperature < 15)
```

→ log /telemetry_alarms

```
SELECT *
FROM 'iot-course/OleksiiPok/telemetry'
WHERE sys_state <> 0
```

→ Log /state_alarms

→ Log /errors

Topic: .../events

```
SELECT *,
       timestamp() AS received_at,
       CAST(clientid() AS String) AS device_id,
       topic(2)    AS student
FROM 'iot-course/OleksiiPok/events'
```

→ DynamoDB /iot_events

→ Log /errors

